# Task 1: Simple E-commerce Store

Express + SQLite + JWT + vanilla JS. Product listing, details page, cart, order processing (server-side price/stock validation in a transaction), register/login, order history.

GitHub repository name: **CodeAlpha_EcommerceStore**

## How to run

```
cd CodeAlpha_EcommerceStore
npm install
npm start        # open http://localhost:3000  (Node 18+)
```

## Overview

Features: product listing, product details, cart (localStorage), checkout with server-side price/stock validation in one DB transaction, register/login (bcrypt + JWT), order history. Tables: users, products, orders, order_items.

Emoji in the UI are omitted from this listing only (the PDF font cannot show them).

## CodeAlpha_EcommerceStore/package.json

```
{ "name": "codealpha-ecommerce", "version": "1.0.0", "main": "server.js", "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.19.2", "better-sqlite3": "*", "bcryptjs": "^2.4.3", "jsonwebtoken": "^9.0.2" } }
```

## CodeAlpha_EcommerceStore/auth.js

```
const jwt = require('jsonwebtoken'), bcrypt = require('bcryptjs');
const SECRET = process.env.JWT_SECRET || 'dev-secret-change-me';
const sign = u => jwt.sign({ id: u.id, username: u.username }, SECRET, { expiresIn: '7d' });
const verify = t => jwt.verify(t, SECRET);
const auth = (req, res, next) => {
  try { req.user = verify((req.headers.authorization || '').replace('Bearer ', '')); next(); }
  catch { res.status(401).json({ error: 'Login required' }); }
};
// Registers POST /api/register and /api/login (needs a `users(id, username, password)` table)
function authRoutes(app, db) {
  app.post('/api/register', (req, res) => {
    const { username, password } = req.body || {};
    if (!/^\w{3,20}$/.test(username || '') || (password || '').length < 6)
      return res.status(400).json({ error: 'Username 3-20 chars (letters/digits/_), password 6+ chars' });
    try {
      const id = db.prepare('INSERT INTO users(username,password) VALUES(?,?)').run(username,
          bcrypt.hashSync(password, 10)).lastInsertRowid;
      res.json({ token: sign({ id, username }), username });
    } catch { res.status(409).json({ error: 'Username already taken' }); }
  });
  app.post('/api/login', (req, res) => {
    const { username, password } = req.body || {};
    const u = db.prepare('SELECT * FROM users WHERE username=?').get(username || '');
    if (!u || !bcrypt.compareSync(password || '', u.password)) return res.status(401).json({ error: 'Wrong username
        or password' });
    res.json({ token: sign(u), username: u.username });
  });
}
module.exports = { sign, verify, auth, authRoutes };
```

## CodeAlpha_EcommerceStore/server.js

```
const express = require('express'), Database = require('better-sqlite3');
const { auth, authRoutes } = require('./auth');
const db = new Database('store.db');
db.exec(`
CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY, username TEXT UNIQUE, password TEXT);
CREATE TABLE IF NOT EXISTS products(id INTEGER PRIMARY KEY, name TEXT, description TEXT, price REAL, image TEXT,
    stock INTEGER);
CREATE TABLE IF NOT EXISTS orders(id INTEGER PRIMARY KEY, user_id INTEGER, total REAL, address TEXT, status TEXT
    DEFAULT 'Placed', created TEXT DEFAULT CURRENT_TIMESTAMP);
CREATE TABLE IF NOT EXISTS order_items(order_id INTEGER, product_id INTEGER, qty INTEGER, price REAL);`);
if (!db.prepare('SELECT COUNT(*) c FROM products').get().c) {
  const ins = db.prepare('INSERT INTO products(name,description,price,image,stock) VALUES(?,?,?,?,?)');
  [['Wireless Earbuds', 'Bluetooth 5.3, 24h battery.', 1499, '', 25], ['Mechanical Keyboard', 'Hot-swappable, blue
      switches.', 3299, '', 10],
   ['Smart Watch', 'Heart-rate and sleep tracking.', 2799, '', 15], ['Backpack', '30L, water resistant, laptop
      sleeve.', 1899, '', 30],
   ['Desk Lamp', 'LED, 3 brightness modes.', 799, '', 40], ['Power Bank', '20,000 mAh, fast charge.', 1699, '',
      20]].forEach(p => ins.run(...p));
}
const app = express(); app.use(express.json()); app.use(express.static('public')); authRoutes(app, db);
app.get('/api/products', (req, res) => res.json(db.prepare('SELECT * FROM products').all()));
```

```
app.get('/api/products/:id', (req, res) => {
  const p = db.prepare('SELECT * FROM products WHERE id=?').get(req.params.id);
  p ? res.json(p) : res.status(404).json({ error: 'Product not found' });
});
// Order processing: validates stock and prices server-side (never trust client prices), atomic transaction
app.post('/api/orders', auth, (req, res) => {
  const { items, address } = req.body || {};
  if (!Array.isArray(items) || !items.length || !(address || '').trim()) return res.status(400).json({ error: 'Items
      and address required' });
  try {
    const orderId = db.transaction(() => {
      let total = 0;
      const rows = items.map(i => {
        const p = db.prepare('SELECT * FROM products WHERE id=?').get(i.id), q = parseInt(i.qty);
        if (!p || !(q > 0) || p.stock < q) throw new Error('Not available: ' + (p ? p.name : 'item ' + i.id));
        total += p.price * q; return [p, q];
      });
      const id = db.prepare('INSERT INTO orders(user_id,total,address) VALUES(?,?,?)').run(req.user.id, total,
          address.trim()).lastInsertRowid;
      rows.forEach(([p, q]) => {
        db.prepare('INSERT INTO order_items VALUES(?,?,?,?)').run(id, p.id, q, p.price);
        db.prepare('UPDATE products SET stock=stock-? WHERE id=?').run(q, p.id);
      });
      return id;
    })();
    res.json({ orderId });
  } catch (e) { res.status(400).json({ error: e.message }); }
});
app.get('/api/orders', auth, (req, res) => {
  const orders = db.prepare('SELECT * FROM orders WHERE user_id=? ORDER BY id DESC').all(req.user.id);
  orders.forEach(o => o.items = db.prepare('SELECT p.name, i.qty, i.price FROM order_items i JOIN products p ON
      p.id=i.product_id WHERE order_id=?').all(o.id));
  res.json(orders);
});
app.listen(process.env.PORT || 3000, () => console.log('Store on http://localhost:3000'));
```

## CodeAlpha_EcommerceStore/public/index.html

```
<!doctype html><html><head><meta charset="utf-8"><meta name="viewport"
    content="width=device-width,initial-scale=1"><title>ShopAlpha</title><link rel="stylesheet"
    href="style.css"></head>
<body><header><b> ShopAlpha</b><span id="nav"></span></header><main id="app"></main>
<script src="common.js"></script>
<script>
let products = [], cart = JSON.parse(localStorage.cart || '[]');
const saveCart = () => { localStorage.cart = JSON.stringify(cart); nav(); };
function nav() {
  $('#nav').innerHTML = `<a onclick="home()">Shop</a><a onclick="showCart()">Cart (${cart.reduce((a, c) => a + c.qty,
      0)})</a>` +
    (token ? `<a onclick="orders()">Orders</a><a onclick="logout()">Logout (${esc(me)})</a>` : `<a
        onclick="$('#app').innerHTML=authBox()">Login</a>`);
}
async function home() {
  products = await api('/products');
  $('#app').innerHTML = '<div class="grid">' + products.map(p => `<div class="card" onclick="detail(${p.id})"
      style="cursor:pointer"><div
      class="big">${p.image}</div><h3>${esc(p.name)}</h3><p>Rs${p.price}</p></div>`).join('') + '</div>';
}
async function detail(id) {
  const p = await api('/products/' + id);
  $('#app').innerHTML = `<div class="card"><a onclick="home()" style="cursor:pointer"><- Back</a><div
      class="big">${p.image}</div><h2>${esc(p.name)}</h2><p>${esc(p.description)}</p><h3>Rs${p.price}</h3><p>${p.stock
      > 0 ? p.stock + ' in stock' : 'Out of stock'}</p><button ${p.stock < 1 ? 'disabled' : ''}
      onclick="add(${p.id})">Add to cart</button></div>`;
}
function add(id) { const c = cart.find(x => x.id === id); c ? c.qty++ : cart.push({ id, qty: 1 }); saveCart();
    alert('Added to cart'); }
function showCart() {
  if (!cart.length) { $('#app').innerHTML = '<div class="card">Cart is empty.</div>'; return; }
  let total = 0;
  const rows = cart.map(c => { const p = products.find(p => p.id === c.id); if (!p) return ''; total += p.price *
      c.qty;
    return `<div class="card">${p.image} <b>${esc(p.name)}</b> - Rs${p.price} x <button class="alt"
        onclick="qty(${c.id},-1)">-</button> ${c.qty} <button class="alt" onclick="qty(${c.id},1)">+</button></div>`;
        }).join('');
  $('#app').innerHTML = rows + `<div class="card"><h3>Total: Rs${total}</h3><textarea id="addr" placeholder="Shipping
      address"></textarea><button onclick="checkout()">Place order</button></div>`;
}
function qty(id, d) { const c = cart.find(x => x.id === id); c.qty += d; if (c.qty < 1) cart = cart.filter(x => x.id
    !== id); saveCart(); showCart(); }
async function checkout() {
  if (!token) return $('#app').innerHTML = authBox();
  try { const r = await api('/orders', 'POST', { items: cart, address: $('#addr').value });
    cart = []; saveCart(); $('#app').innerHTML = `<div class="card"> Order #${r.orderId} placed!</div>`;
  } catch (e) { alert(e.message); }
}
async function orders() {
  const os = await api('/orders');
```

```
  $('#app').innerHTML = os.map(o => `<div class="card"><b>Order #${o.id}</b> - ${esc(o.status)} -
      Rs${o.total}<br>${o.items.map(i => `${esc(i.name)} x ${i.qty}`).join(',
      ')}<br><small>${esc(o.address)}</small></div>`).join('') || '<div class="card">No orders yet.</div>';
}
nav(); home().then(() => { });
</script></body></html>
```

## CodeAlpha_EcommerceStore/public/common.js

```
const $ = s => document.querySelector(s);
let token = localStorage.token || '', me = localStorage.me || '';
async function api(path, method = 'GET', body) {
  const r = await fetch('/api' + path, { method, headers: { 'Content-Type': 'application/json', Authorization:
      'Bearer ' + token }, body: body ? JSON.stringify(body) : undefined });
  const d = await r.json().catch(() => ({}));
  if (r.status === 401 && token) { localStorage.clear(); location.reload(); }
  if (!r.ok) throw new Error(d.error || 'Something went wrong');
  return d;
}
async function authSubmit(kind) {
  try { const d = await api('/' + kind, 'POST', { username: $('#u').value, password: $('#p').value });
    token = localStorage.token = d.token; me = localStorage.me = d.username; location.reload();
  } catch (e) { alert(e.message); }
}
function logout() { localStorage.removeItem('token'); localStorage.removeItem('me'); location.reload(); }
const esc = s => String(s ?? '').replace(/[&<>"']/g, c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;',
    "'": '&#39;' }[c]));
const authBox = () => `<div class="card narrow"><h3>Login / Register</h3><input id="u" placeholder="username"><input
    id="p" type="password" placeholder="password (6+ chars)"><button onclick="authSubmit('login')">Login</button>
    <button class="alt" onclick="authSubmit('register')">Register</button></div>`;
```

## CodeAlpha_EcommerceStore/public/style.css

```
*{box-sizing:border-box}body{margin:0;font-family:system-ui,sans-serif;background:#f4f5f7;color:#222}
header{display:flex;justify-content:space-between;align-items:center;padding:12px
    18px;background:#1f2937;color:#fff;position:sticky;top:0;z-index:5}
header a{color:#cbd5e1;margin-left:14px;text-decoration:none;cursor:pointer}header a:hover{color:#fff}
main{max-width:1000px;margin:18px auto;padding:0 12px}
.card{background:#fff;border-radius:10px;padding:14px;margin-bottom:12px;box-shadow:0 1px 3px
    #0002}.narrow{max-width:360px;margin:40px auto}
input,textarea,select{width:100%;padding:9px;margin:5px 0;border:1px solid #cbd5e1;border-radius:6px;font:inherit}
button,.btn{background:#2563eb;color:#fff;border:0;padding:8px
    14px;border-radius:6px;cursor:pointer;font:inherit;display:inline-block}
button.alt,.btn.alt{background:#64748b}button:hover{opacity:.9}
.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(210px,1fr));gap:12px}.big{font-size:56px;text-align:c
    enter}
.cols{display:grid;grid-template-columns:repeat(3,1fr);gap:12px}@media(max-width:700px){.cols{grid-template-columns:1f
    r}}
.cm{background:#f1f5f9;border-radius:6px;padding:6px 9px;margin:5px 0;font-size:14px}
#toast div{position:fixed;right:14px;bottom:14px;background:#111;color:#fff;padding:10px 14px;border-radius:8px}
```